Slides & Docs

On this page

01: Første Image

Formål: Forstå hvad en Dockerfile er, og hvordan et image bygges og køres.

Opgave:

Opret en simpel webapplikation, vælg mellem:

- Node: Express server på port 3000
- C#: Minimal API på port 5000

Skriv en Dockerfile der:

- Bruger et officelt base image (se https://hub.docker.com)
- Kopierer din applikation ind i containeren
- Installerer dependencies
- Starter applikationen

Opgaven er løst når:

- Du kan bygge et Docker image fra din Dockerfile uden fejl
- Du kan køre en container baseret på dit image, og se din applikation
- Image ikke er tagget som "latest"

02: Struktur og layer forståelse

Formål:

Forstå hvordan layers og caching fungerer

Opgave:

Med udgangspunkt i Dockerfile fra øvelse 1

1. Byg dit image og noter build-tiden
2. Lav en ændring i din applikation (fx tilføj en log linje)
3. Byg dit image igen og noter build-tiden
4. Observer hvilke layers der genbruges og hvilke der bygges på ny

Opgaven er løst når:

- Du kan forklare hvorfor nogle layers genbruges og andre ikke

03: Docker Ignore

Formål

Forstå build context og hvorfor det påvirker performance

Opgave:

1. Bug image uden `.dockerignore` og noter build-tiden
2. Opret en `.dockerignore` fil og tilføj `node_modules` (eller `bin/obj` for C#)
3. Byg image igen og noter build-tiden

Research:

- Hvad er build context?
- Hvorfor påvirker det caching?

Opgaven er løst når:

- Du kan forklare hvad build context er og hvorfor det påvirker caching

04: Optimering af Dockerfile

Formål:

Forstå image størrelse og best pracices

Opgave:

Analyser dit image

bash

```bash
$ docker images
```

Research:

- Hvad påvirker image størrelse?
- Hvad indeholder base image?

Optimer:

- Kan du gøre dit image mindre ved at ændre rækkefølgen af kommandoer i Dockerfile, eller kombinere RUN kommandoer?
- Kan du bruge et mindre base image?

Opgaven er løst når:

- Du kan forklare hvad der påvirker image størrelse
- Du har optimeret dit image og kan forklare hvad du har gjort og hvorfor
- Applikation virker stadig

05: CMD vs ENTRYPOINT

Formål:

Forstå forskellen på runtime behaviour

Opgave:

Lav to versioner

1. Med kun `CMD`
2. Med `ENTRYPOINT` og `CMD`

Research:

- HVad "overrides" af hvad?
- Hvad sker der med argumenter?

Opgaven er løst når:

- Kommando override virker forskelligt mellem versionerne
- Forklaring er teknisk korrekt

06: ARG vs ENV

Formål:

Forstå forskellen på build-time og runtime variabler

Opgave:

Tilføj:

- En build variabel
- En runtime env variabel

Test:

- Brug `--build-arg`
- Brug `-e` ved docker run

Research:

- Kan ARG ses i kørende container
- Kan ENV ses i image history

Opgaven er løst når:

- Du kan forklare forskellen på ARG og ENV
- Du kan demonstrere hvordan de bruges og hvad deres scope er

07: Større opgave

Formål:

Anvende det lærte i en større opgave

Opgave:

Udvikl' en simpel webapplikation

- Node: Express API
- C#: Minimal API

Applikationen skal:

- Køre som webserver
- Kunne gemme og hente data

- Data må ikke gå tabt når containeren stoppes eller slettes

Data må gemmes:

- I fil (JSON)
- I SQLite database

Serveren skal kunne tilgåes på relevant port

Opgaven er løst når:

- Applikationen virker som forventet
- Data persistens virker selvom containeren stoppes og slettes
- Image er optimeret og ikke unødvendigt stort
- Dockerfile er veldokumenteret og følger best practices